INTERVIEW NOTEBOOK · NETFLIX · DATA ENGINEERING

The interview is a *conversation*, not a quiz.

Netflix interviewers explicitly say: *"we care more about your logic than your memorization of SQL engine-specific quirks."* This notebook is built around that premise — three deep sections (Modeling, SQL, Programming), each with the patterns Netflix actually probes, worked solutions, and the questions you should be asking **them**.

CONTENTS

§ 01 — CALIBRATION

The Netflix bar, and what they're *actually* probing.

Before any code, understand the rubric. Netflix DE rounds aren't LeetCode contests. They're structured conversations where the interviewer is silently scoring four things on every answer:

01 · CLARIFICATION REFLEX

Do you ask sharp questions before coding? "What's the grain?" "Is this batch or stream?" "Are user_ids stable?" Silence = junior signal.

02 · TRADE-OFF ARTICULATION

Every answer has a cost. Star schema vs. wide table. Window function vs. self-join. Can you name the trade and pick one with reasoning?

03 · FAILURE THINKING

What happens when this pipeline runs at 3am, the upstream is late, and the watermark is wrong? Senior candidates volunteer this.

04 · COMMUNICATION UNDER PRESSURE

Narrate as you code. "I'm writing a CTE first to isolate qualified sessions." Silent typing = unscoreable.

> THE RECRUITER'S QUOTE, DECODED
>
> *"We care more about your logic than your memorization of SQL engine-specific quirks."* — This means: **don't panic if you forget exact syntax.** Pseudocode the logic, name the function family ("I'd use a window function with PARTITION BY user_id ORDER BY ts, then LAG to compute gaps"), and the interviewer will nod. **Spending 2 minutes recalling whether it's `DATE_DIFF` or `DATEDIFF` is a waste.** Say "in Snowflake syntax this is roughly..." and move on.

What Netflix DE rounds typically look like

- **Data Modeling (45–60 min):** One open-ended scenario. They want to see you go from ambiguous prompt → clarifying questions → conceptual model → physical model → indexing/partitioning → "how does this break?"
- **SQL (45–60 min):** 3–6 questions of escalating difficulty against a shared schema. Expect window functions, sessionization, and at least one "data quality" or "deduplication" question.
- **Programming (45–60 min):** CodeSignal-style. 1–2 problems. Then 15–20 min deep-dive on streaming, data systems, or a project from your resume.

---

§ 02 — DATA MODELING

Modeling is a *conversation* — five moves, every time.

The modeling round is the hardest to fake and the easiest to fail by jumping

straight to tables. Use the same five-move sequence on any prompt — viewing history, billing, A/B experiments, content metadata, payouts — and you will look senior even when the domain is unfamiliar.

The five-move sequence

1. **Clarify the use cases.** "Who reads this? What questions does it answer? What's the SLA — minutes, hours, days?" Don't model anything until you have 3–5 named consumers.
2. **Pin the grain.** One sentence per fact table: "One row per user per title per session." If you can't say this cleanly, the model isn't ready.
3. **Conceptual → logical → physical.** Entities and relationships first. Then attributes and types. Then partitioning, clustering, and SCD strategy.
4. **SCDs and time.** Which dimensions change? How do you preserve history — Type 1 overwrite, Type 2 versioned rows, or Type 6 hybrid? Most candidates skip this and lose senior signal.
5. **Stress test.** "How does this break?" Late-arriving facts, time zones, GDPR delete requests, schema drift, hot partitions. **Volunteer at least two failure modes** — don't wait to be asked.

> THE GRAIN TRAP
>
> The single most common mid-level mistake: writing `fact_views` with no clear grain, then later realizing one user can have multiple device sessions per title and the row counts don't add up. **Say the grain out loud before writing a single column.**

Worked example — the prompt you should expect

MODEL · 01 OPEN-ENDED · 45 MIN

"Design a data model to support Netflix's viewing history. Stakeholders include the recommendations team, the content team (which titles to renew), and finance (royalty payouts to studios)."

▸ WALK-THROUGH — THE WAY TO ANSWER THIS LIVE

Other modeling prompts you should rehearse

MODEL · 02 BILLING / FINANCE

"Design a model for subscription billing — handle plan changes mid-cycle, prorations, gift codes, failed payments, win-backs."

▸ HINTS

MODEL · 03 A/B EXPERIMENTATION

"Design a model to support A/B test analysis at Netflix scale — thousands of concurrent experiments, ability to compute lift on any metric for any test."

▸ HINTS

MODEL · 04 CONTENT METADATA

"Model Netflix's content catalog — titles have many languages of audio/subtitles, multiple regional availability windows, ratings differ by country, and there's a hierarchy (series → season → episode)."

▸ HINTS

The vocabulary you must use fluently

If you hesitate on any of these terms, drill them before the interview. Each one is a lever you'll reach for during the modeling round.

- **Grain** — the meaning of one row. Stated as a sentence.
- **SCD Type 1 / 2 / 6** — overwrite vs. versioned-rows vs. hybrid (current + history columns).
- **Conformed dimension** — same dim used by multiple fact tables (dim_date, dim_geography). Enables cross-fact analysis.
- **Bridge / factless fact** — many-to-many resolver, or an event-occurrence table with no measure.
- **Late-arriving dimension** — fact arrives before the dim is loaded. Use a placeholder SK (-1) and back-fill, or block the load.
- **Slowly-changing fact** — restating financials. Append corrections; never UPDATE in place.
- **Idempotent load** — re-running yesterday's batch produces the same result. Always achievable via MERGE on a natural key.
- **Watermark** — the latest event time you trust as "complete enough" to materialize. Stream concept; relevant for late data.

§ 03 — SQL

SQL — In flight, at rest, and the *seven patterns* that cover 80% of Netflix questions.

Netflix's SQL round is engine-agnostic by design. Master these seven patterns and you can adapt any of them to whatever schema they put in front of you. Every solution below is written in standard SQL with notes on Snowflake / BigQuery / Spark dialects where they matter.

> READING THE ROOM
>
> If the interviewer says "write SQL," they want runnable SQL. If they say "how would you compute…", you can lead with pseudocode and CTEs first, then commit to syntax. Always narrate: "Step one, get the qualified sessions. Step two, rank within user. Step three, filter rank=1." This is what they're actually grading.

Pattern 1 — Top-N per group

SQL · 01 WINDOW FUNCTIONS

From `fact_viewing_session`, return each profile's top 3 most-watched titles by total watch_seconds in the last 30 days.

▾ SOLUTION

```
WITH agg AS (
```

```
  SELECT
    profile_id,
    title_id,
    SUM(watch_seconds) AS total_seconds
  FROM fact_viewing_session
  WHERE session_start_ts >= CURRENT_DATE - INTERVAL '30 days
  GROUP BY profile_id, title_id
),
ranked AS (
  SELECT
    profile_id,
    title_id,
    total_seconds,
    ROW_NUMBER() OVER (
      PARTITION BY profile_id
      ORDER BY total_seconds DESC, title_id ASC
    ) AS rn
  FROM agg
)
SELECT profile_id, title_id, total_seconds
FROM ranked
WHERE rn <= 3
ORDER BY profile_id, rn;
```

TALKING POINTS

- **ROW_NUMBER vs RANK vs DENSE_RANK** – pick ROW_NUMBER when you need exactly 3 rows. RANK when ties should both be included (you might get 4). Mention the trade.
- **Tie-breaker:** always add a deterministic secondary sort (`title_id ASC`) – without it, results are non-deterministic and tests will flake.
- **Snowflake shortcut:** use `QUALIFY ROW_NUMBER() OVER (...) <= 3` instead of the second CTE. Mention it; don't depend on it.
- **Performance:** if there are 10B sessions, the GROUP BY is the expensive step – partition pruning on session_start_ts is

doing the heavy lifting.

Pattern 2 — Sessionization (gaps and islands)

SQL · 02 WINDOW · GAPS & ISLANDS

Given **`fact_play_event(profile_id, ts, event_type)`**, define a "binge session" as consecutive events with no more than 30 minutes between any two. Return one row per binge session per profile with start_ts, end_ts, event_count.

▾ SOLUTION

```
WITH with_prev AS (
  SELECT
    profile_id,
    ts,
    event_type,
    LAG(ts) OVER (PARTITION BY profile_id ORDER BY ts) AS pr
  FROM fact_play_event
),
flagged AS (
  SELECT
    profile_id, ts, event_type,
    CASE
      WHEN prev_ts IS NULL
        OR TIMESTAMPDIFF('second', prev_ts, ts) > 1800
      THEN 1 ELSE 0
    END AS is_new_session
  FROM with_prev
),
sessionized AS (
  SELECT
    profile_id, ts, event_type,
```

```
    SUM(is_new_session) OVER (
      PARTITION BY profile_id
      ORDER BY ts
      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS session_num
  FROM flagged
)
SELECT
  profile_id,
  session_num,
  MIN(ts) AS session_start,
  MAX(ts) AS session_end,
  COUNT(*) AS event_count
FROM sessionized
GROUP BY profile_id, session_num
ORDER BY profile_id, session_start;
```

TALKING POINTS

- This is the canonical **gaps-and-islands** pattern. Three steps: LAG to compute gap, flag boundaries, cumulative SUM to assign session IDs. Memorize the shape.
- The `is_new_session` flag is 1 when the gap exceeds threshold OR when there's no prior row. Cumulative SUM turns boundary flags into stable session numbers.
- **Dialect note:** `TIMESTAMPDIFF` is Snowflake/MySQL. BigQuery uses `TIMESTAMP_DIFF(ts, prev_ts, SECOND)`. Postgres: `EXTRACT(EPOCH FROM (ts - prev_ts))`. Don't get stuck — say "in *X* dialect this is..." and continue.
- **Senior signal:** mention that for a 30-day stream, you'd run this incrementally with a backfill window of (30min × 2) at the boundary to avoid splitting sessions across batches.

Pattern 3 — Funnels and conversion

SQL · 03 FUNNEL · CONDITIONAL AGGREGATION

From **`fact_signup_event(profile_id, step_name, ts)`** where step_name ∈ {landing, plan_select, payment_info, confirm}, compute conversion rate at each step.

▾ SOLUTION

```
WITH per_user AS (
  SELECT
    profile_id,
    MAX(CASE WHEN step_name = 'landing'      THEN 1 END) AS
    MAX(CASE WHEN step_name = 'plan_select'  THEN 1 END) AS
    MAX(CASE WHEN step_name = 'payment_info' THEN 1 END) AS
    MAX(CASE WHEN step_name = 'confirm'      THEN 1 END) AS
  FROM fact_signup_event
  GROUP BY profile_id
)
SELECT
  SUM(hit_landing)  AS landing_n,
  SUM(hit_plan)     AS plan_n,
  SUM(hit_pay)      AS pay_n,
  SUM(hit_confirm)  AS confirm_n,
  SUM(hit_plan)::FLOAT    / NULLIF(SUM(hit_landing), 0) AS c
  SUM(hit_pay)::FLOAT     / NULLIF(SUM(hit_plan),    0) AS c
  SUM(hit_confirm)::FLOAT / NULLIF(SUM(hit_pay),     0) AS c
  SUM(hit_confirm)::FLOAT / NULLIF(SUM(hit_landing), 0) AS o
FROM per_user;
```

TALKING POINTS

- **Conditional aggregation** (CASE inside aggregate) is faster than self-joining the table four times. Always preferred for funnels.

- **Strict vs. inclusive funnel:** the above counts users who hit each step *at any point*. For a strict funnel (must complete previous steps in order), add timestamp checks: `MAX(CASE WHEN step='plan' AND ts > landing_ts THEN 1 END)`.
- **NULLIF** on the denominator prevents division-by-zero. Always.
- **Senior signal:** mention that real funnel analysis has a *time window per step* — "did they reach plan_select within 7 days of landing?" — and propose using a windowed self-join or a single-pass FIRST_VALUE per step.

Pattern 4 — Deduplication / data quality

SQL · 04 DEDUP · IDEMPOTENCY

An upstream Kafka consumer occasionally double-writes events. Given **`fact_play_event_raw(event_id, profile_id, ts, event_type, ingest_ts)`** where **`event_id`** should be unique, return a clean dataset keeping the *latest ingested* version of each event_id.

▾ SOLUTION

```
WITH ranked AS (
  SELECT *,
    ROW_NUMBER() OVER (
      PARTITION BY event_id
      ORDER BY ingest_ts DESC
    ) AS rn
```

```
  FROM fact_play_event_raw
)
SELECT event_id, profile_id, ts, event_type, ingest_ts
FROM ranked
WHERE rn = 1;
```

TALKING POINTS

- **Why ROW_NUMBER over DISTINCT:** DISTINCT collapses on all columns. If anything other than event_id differs (e.g., a corrected ts), DISTINCT keeps both rows. ROW_NUMBER picks one deterministically.
- **Why ingest_ts not event_ts:** the producer's clock may be wrong. Trust the broker's ingest time for "which write wins."
- **Senior signal — make it idempotent at write time:** use `MERGE INTO target USING source ON event_id WHEN MATCHED AND source.ingest_ts > target.ingest_ts THEN UPDATE...` instead of cleaning at read time.
- **Volunteer the alternative:** if events are truly immutable, a `QUALIFY ROW_NUMBER() = 1` (Snowflake) or HASH-based deduplication at ingest is cheaper than a daily scan.

Pattern 5 — Cohort retention

SQL · 05 COHORT · SELF-JOIN

Compute weekly retention for the cohort of users who signed up in week W: what % of week-W signups had any qualified view in week W+1, W+2, …, W+12?

▾ SOLUTION

```sql
WITH cohort AS (
  SELECT
    profile_id,
    DATE_TRUNC('week', signup_ts) AS cohort_week
  FROM dim_profile
),
activity AS (
  SELECT DISTINCT
    profile_id,
    DATE_TRUNC('week', session_start_ts) AS activity_week
  FROM fact_viewing_session
  WHERE qualified_view_flag = TRUE
),
joined AS (
  SELECT
    c.cohort_week,
    c.profile_id,
    a.activity_week,
    DATEDIFF('week', c.cohort_week, a.activity_week) AS week
  FROM cohort c
  LEFT JOIN activity a
    ON c.profile_id = a.profile_id
   AND a.activity_week >= c.cohort_week
),
sized AS (
  SELECT cohort_week, COUNT(DISTINCT profile_id) AS cohort_s
  FROM cohort
  GROUP BY cohort_week
)
SELECT
  j.cohort_week,
  j.week_offset,
  COUNT(DISTINCT j.profile_id) AS retained,
  s.cohort_size,
  COUNT(DISTINCT j.profile_id)::FLOAT / s.cohort_size AS ret
FROM joined j
JOIN sized s ON j.cohort_week = s.cohort_week
WHERE j.week_offset BETWEEN 0 AND 12
GROUP BY j.cohort_week, j.week_offset, s.cohort_size
ORDER BY j.cohort_week, j.week_offset;
```

TALKING POINTS

- **Cohort definition is a choice.** Calendar week vs. rolling 7-day; first-event vs. signup. State your choice.
- The LEFT JOIN keeps cohort users with zero activity, which is essential for the denominator.
- **Senior signal:** for production, materialize this as a daily snapshot table and let BI tools pivot — don't recompute from raw on every dashboard load.

Pattern 6 — Running totals and pacing

SQL · 06 WINDOW · CUMULATIVE

For each title, compute cumulative qualified views by day since release. Flag any day where the title's cumulative views crossed 1M.

▾ SOLUTION

```
WITH daily AS (
  SELECT
    title_id,
    DATE(session_start_ts) AS view_date,
    COUNT(*) AS daily_views
  FROM fact_viewing_session
  WHERE qualified_view_flag = TRUE
  GROUP BY title_id, DATE(session_start_ts)
),
cumulative AS (
  SELECT
    title_id,
    view_date,
```

```
    daily_views,
    SUM(daily_views) OVER (
      PARTITION BY title_id
      ORDER BY view_date
      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS cumulative_views,
    SUM(daily_views) OVER (
      PARTITION BY title_id
      ORDER BY view_date
      ROWS BETWEEN 1 PRECEDING AND 1 PRECEDING
    ) AS prev_cumulative
  FROM daily
)
SELECT
  title_id,
  view_date,
  daily_views,
  cumulative_views,
  CASE
    WHEN cumulative_views >= 1000000
     AND (prev_cumulative IS NULL OR prev_cumulative < 10000
    THEN TRUE ELSE FALSE
  END AS crossed_1m_today
FROM cumulative
ORDER BY title_id, view_date;
```

TALKING POINTS

- The `ROWS BETWEEN ... PRECEDING` trick to get "previous cumulative" without LAG is worth showing — it works even if there are gaps in the date series.
- **Edge case to volunteer:** what if a title has no views on day N? You'd want to densify with `generate_series` or a date spine before the cumulative — otherwise the "crossed today" flag can fire on the wrong day.

Pattern 7 — DAU / MAU and HyperLogLog

SQL · 07 APPROXIMATE AGGREGATION

Compute DAU and MAU for the last 30 days. Discuss how to make this efficient at petabyte scale.

▾ SOLUTION

```sql
-- Exact, expensive:
SELECT
  DATE(session_start_ts) AS d,
  COUNT(DISTINCT profile_id) AS dau
FROM fact_viewing_session
WHERE session_start_ts >= CURRENT_DATE - 30
GROUP BY 1;

-- Approximate, cheap (Snowflake / BigQuery):
SELECT
  DATE(session_start_ts) AS d,
  APPROX_COUNT_DISTINCT(profile_id) AS dau_approx
FROM fact_viewing_session
WHERE session_start_ts >= CURRENT_DATE - 30
GROUP BY 1;

-- HLL sketches for rollup-friendly storage:
CREATE TABLE dau_sketches AS
SELECT
  DATE(session_start_ts) AS d,
  HLL_ACCUMULATE(profile_id) AS sketch
FROM fact_viewing_session
GROUP BY 1;

-- Then MAU is a cheap merge of 30 sketches:
SELECT HLL_ESTIMATE(HLL_COMBINE(sketch)) AS mau_approx
FROM dau_sketches
WHERE d >= CURRENT_DATE - 30;
```

TALKING POINTS

- **This is a senior signal.** Mention HyperLogLog sketches by name. Explain that they're additive (you can merge sketches

without re-scanning raw events), which makes weekly/monthly/yearly rollups trivial.

- Typical accuracy: ~1.5% error at default precision. Acceptable for product analytics, not for finance.
- **BigQuery**: `HLL_COUNT.INIT / MERGE / EXTRACT` . **Snowflake**: `HLL_ACCUMULATE / HLL_COMBINE / HLL_ESTIMATE` . Names differ; concept is identical.

"In flight" SQL — what they probably mean

The recruiter mentioned "data in flight and at rest." For SQL, that almost certainly means:

- **At rest:** standard analytical queries against materialized tables. Patterns 1–7 above.
- **In flight:** SQL-on-streams (Flink SQL, Spark Structured Streaming, ksqlDB). They might ask *"how would you compute a 5-minute tumbling window of qualified views per title from a Kafka topic?"* Be ready to write Flink SQL or pseudocode it.

```
-- Flink SQL — tumbling window over a Kafka source
CREATE TABLE play_events (
  profile_id BIGINT,
  title_id BIGINT,
  watch_seconds INT,
  event_ts TIMESTAMP(3),
  WATERMARK FOR event_ts AS event_ts - INTERVAL '30' SECOND
) WITH ('connector' = 'kafka', ...);

SELECT
  title_id,
  TUMBLE_START(event_ts, INTERVAL '5' MINUTE) AS window_start,
```

```
  COUNT(*) AS events,
  COUNT(DISTINCT profile_id) AS unique_viewers
FROM play_events
WHERE watch_seconds >= 120
GROUP BY title_id, TUMBLE(event_ts, INTERVAL '5' MINUTE);
```

Vocabulary to drop: watermark, event-time vs. processing-time, tumbling vs. hopping vs. sliding windows, late-data tolerance, exactly-once semantics, checkpoint interval, state backend (RocksDB).

---

§ 04 — PROGRAMMING

CodeSignal — fluency over *cleverness*.

The recruiter explicitly said: *"we care more about coding fluency than perfect solutioning."* Translation: a working O(n²) solution that you can explain and improve beats a clever O(n log n) you can't articulate. Here are the patterns Netflix actually tests, with full Python solutions.

CODESIGNAL MECHANICS

CodeSignal runs your code against hidden test cases. You see pass/fail per test, not the inputs. **Always start by writing a brute-force solution that passes the first 2–3 tests**, then optimize. A partial solution scores higher than a perfect solution you didn't finish. Keep one eye on the timer.

Pattern 1 — Stream processing (the Netflix favorite)

CODE · 01 HEAP · SLIDING WINDOW

You receive a stream of viewing events `(user_id, title_id, watch_seconds, ts)`. At any point, return the top-K most-watched titles in the last 1 hour. Implement `add_event(event)` and `top_k(k)`.

▾ SOLUTION

```
from collections import defaultdict, deque
import heapq

class TopKWatchedStream:
    def __init__(self, window_seconds=3600):
        self.window = window_seconds
        self.events = deque()  # (ts, title_id, watch_secona
        self.totals = defaultdict(int)  # title_id -> secona

    def add_event(self, event):
        ts, title_id, watch = event['ts'], event['title_id']
        self.events.append((ts, title_id, watch))
        self.totals[title_id] += watch
        self._evict(ts)

    def _evict(self, now):
        cutoff = now - self.window
        while self.events and self.events[0][0] < cutoff:
            old_ts, old_title, old_watch = self.events.pople
            self.totals[old_title] -= old_watch
            if self.totals[old_title] <= 0:
                del self.totals[old_title]

    def top_k(self, k):
        # For small K and many titles, heap.nlargest is best
        return heapq.nlargest(k, self.totals.items(), key=la
```

TALKING POINTS

- **Why deque + dict, not a sorted structure:** add is O(1), eviction amortized O(1). top_k is O(n log k) but called rarely.
- **If top_k is called constantly**, switch to a max-heap with lazy

deletion (push (-total, title) on every update; pop stale entries on read).

- **Distributed version:** partition by title_id across workers, each maintains its own top-K, then merge. Mention Spark Streaming's `updateStateByKey` or Flink's keyed state.
- **Out-of-order events:** the `_evict` uses "now" from the latest event. If events arrive out of order, you need a watermark, not last-seen-ts.

Pattern 2 — Sessionization in code

CODE · 02 ITERATION · GROUPING

Given a list of events `[(user_id, ts), ...]`, group into sessions where consecutive events for the same user are within `gap` seconds. Return list of sessions `[(user_id, session_start, session_end, event_count)]`.

▾ SOLUTION

```
from collections import defaultdict

def sessionize(events, gap=1800):
    # Group by user, sort each user's events
    by_user = defaultdict(list)
    for uid, ts in events:
        by_user[uid].append(ts)

    sessions = []
    for uid, ts_list in by_user.items():
        ts_list.sort()
```

```
        session_start = ts_list[0]
        session_end = ts_list[0]
        count = 1
        for i in range(1, len(ts_list)):
            if ts_list[i] - session_end <= gap:
                session_end = ts_list[i]
                count += 1
            else:
                sessions.append((uid, session_start, session
                session_start = ts_list[i]
                session_end = ts_list[i]
                count = 1
        sessions.append((uid, session_start, session_end, co
    return sessions
```

TALKING POINTS

- O(n log n) total — dominated by per-user sort. If events are pre-sorted (typical from a stream), single pass is O(n).
- **Memory:** O(n). For large data, stream-process by user_id partition (Spark / Flink) — each worker only holds one user's events.
- **Edge case to volunteer:** what if two events have the same ts? Decide if they're the same session (ts ≤ gap is true: yes). Always state the assumption.

Pattern 3 — Top-K from a stream (frequency)

CODE · 03 HEAP · COUNTER

Given a list of **`(title_id, watch_seconds)`** tuples, return the K title_ids with the highest total watch time. Optimize for memory if there are billions of events but

only millions of distinct titles.

▾ SOLUTION

```
from collections import defaultdict
import heapq

def top_k_titles(events, k):
    totals = defaultdict(int)
    for title_id, watch in events:
        totals[title_id] += watch
    # nlargest is O(n log k); faster than sorting all when k
    return heapq.nlargest(k, totals.items(), key=lambda x: x

# Memory-efficient version: maintain heap as you iterate
def top_k_streaming(events, k):
    totals = defaultdict(int)
    for title_id, watch in events:
        totals[title_id] += watch

    heap = []  # min-heap of (total, title_id), size k
    for title, total in totals.items():
        if len(heap) < k:
            heapq.heappush(heap, (total, title))
        elif total > heap[0][0]:
            heapq.heapreplace(heap, (total, title))
    return sorted(heap, reverse=True)
```

TALKING POINTS

- **nlargest vs manual heap:** identical asymptotic, but manual heap is O(k) memory whereas nlargest builds the full list first. Memory matters at scale.
- **If titles don't fit in memory:** Count-Min Sketch + a top-K heap. Approximate but constant memory.
- **Distributed:** map-reduce — each partition computes local top-K, reducer merges to global top-K. Top-K is associative for sums; this works.

Pattern 4 — Two-pointer / merge

CODE · 04 TWO-POINTER

Given two sorted streams of timestamps (e.g., login events from two regions), merge them into a single chronological stream *without* loading either fully into memory.

▾ SOLUTION

```
def merge_streams(stream_a, stream_b):
    # Both are iterators yielding sorted timestamps
    a = iter(stream_a)
    b = iter(stream_b)
    ta = next(a, None)
    tb = next(b, None)

    while ta is not None and tb is not None:
        if ta <= tb:
            yield ta
            ta = next(a, None)
        else:
            yield tb
            tb = next(b, None)

    # Drain whichever is left
    while ta is not None:
        yield ta
        ta = next(a, None)
    while tb is not None:
        yield tb
        tb = next(b, None)
```

TALKING POINTS

- **Generator pattern:** O(1) memory, processes streams of any

size.

- **K-way merge** (more than 2 streams): use a min-heap of (next_ts, stream_id). This is exactly Kafka consumer-group merging.
- **Mention `heapq.merge()`** — Python's stdlib does this. Knowing it exists is a fluency signal; implementing it manually shows you understand the mechanic.

Pattern 5 — Hash map / counter (the easy money)

CODE · 05 HASH MAP

Given a list of `(user_id, title_id)` pairs (one per view), find pairs of users who watched at least 3 titles in common.

▾ SOLUTION

```
from collections import defaultdict
from itertools import combinations

def users_with_overlap(views, threshold=3):
    # Step 1: title -> set of users who watched it
    title_to_users = defaultdict(set)
    for uid, tid in views:
        title_to_users[tid].add(uid)

    # Step 2: count co-occurrences via pair enumeration per
    pair_counts = defaultdict(int)
    for users in title_to_users.values():
        for u1, u2 in combinations(sorted(users), 2):
            pair_counts[(u1, u2)] += 1
```

```
    return [pair for pair, c in pair_counts.items() if c >=
```

TALKING POINTS

- **Complexity warning:** if a single title has M users, that's M^2 pairs. For Netflix-scale (popular titles can have millions of viewers), this is intractable. **Volunteer this.**
- **Better approach at scale:** MinHash / LSH (locality-sensitive hashing) — approximate set intersection without enumerating pairs. O(n) instead of $O(n^2)$.
- **Or:** per-user title set, then sample candidate pairs by shared "popular" titles only. Trade exactness for tractability.

Pattern 6 — LRU cache (the system-design crossover)

CODE · 06 ORDEREDDICT · DOUBLY-LINKED LIST

Implement an LRU cache for content metadata. `get(title_id)` returns the metadata or -1, `put(title_id, metadata)` inserts/updates. Both must be O(1).

▾ SOLUTION (PYTHONIC)

```
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity):
        self.cache = OrderedDict()
        self.capacity = capacity

    def get(self, key):
```

```
        if key not in self.cache:
            return -1
        self.cache.move_to_end(key)
        return self.cache[key]

    def put(self, key, value):
        if key in self.cache:
            self.cache.move_to_end(key)
        self.cache[key] = value
        if len(self.cache) > self.capacity:
            self.cache.popitem(last=False)
```

TALKING POINTS

- **OrderedDict is backed by a doubly-linked list + hash map.** If asked to implement from scratch, write the DLL version (Node class with prev/next, plus a dict for O(1) lookup).
- **Why this matters at Netflix:** CDN edge caches, ML feature stores, content metadata caches all use LRU or LFU variants. Be ready to discuss when LFU > LRU (recurring access patterns) and when neither works (one-shot streams — use TTL).

Coding under pressure — the script you should follow

1. **Restate the problem.** "So I'm getting a list of tuples and need to return..." 30 seconds. Catches misunderstanding early.
2. **Walk through one example by hand.** "If input is [(1,A), (1,B), (2,A)], expected output is..." This forces you to confirm edge cases.
3. **Brute-force first.** "I'll start with the obvious $O(n^2)$ approach, then optimize." Get something passing.
4. **Optimize with reasoning.** "The bottleneck is the inner loop. If I

precompute a dict, that becomes O(1)..."

5. **Test mentally.** Run your code on the example you wrote. Catch off-by-ones before the timer does.
6. **Discuss complexity.** Time and space, both. State them explicitly even if not asked.

---

§ 05 — DATA IN FLIGHT

Streaming concepts — the vocabulary that signals *seniority*.

Netflix runs Keystone (a massive Kafka pipeline) and uses Flink heavily. Even if you don't write a line of Flink in the interview, knowing this vocabulary is what separates a senior DE from a SQL specialist.

EVENT-TIME VS. PROCESSING-TIME

Event-time is when the event occurred (user pressed play). **Processing-time** is when your pipeline saw it. Production analytics always use event-time; processing-time is a fallback for pipelines without reliable timestamps.

WATERMARK

The pipeline's promise: "I have seen all events with timestamp ≤ W." Watermarks let you close windows and emit results despite out-of-order arrivals. The trade: how long to wait (lateness budget) vs. how fresh the output is.

TUMBLING / HOPPING / SLIDING WINDOWS

Tumbling: non-overlapping fixed buckets (every 5 min, distinct). **Hopping:** overlapping fixed buckets (every 1 min, looking back 5 min).
Sliding/session: dynamic boundaries based on event gaps (sessionization).

EXACTLY-ONCE VS AT-LEAST-ONCE

At-least-once: events may be duplicated on retry. Cheap. Requires idempotent consumers. **Exactly-once:** Flink's checkpoint + 2PC, or Kafka transactions. Expensive in latency and state. Most Netflix pipelines use at-least-once + idempotent writes.

BACKPRESSURE

When a downstream operator is slower than upstream, the system pushes back. Flink does this natively. Without backpressure handling, you get OOMs or data loss. Mention it.

CHECKPOINTS & STATE BACKENDS

Flink periodically snapshots operator state to durable storage (S3). On failure, restore from the last checkpoint. **State backend:** RocksDB (disk-based, large state) vs. heap (small, fast). The choice is a real interview discussion.

LAMBDA VS. KAPPA ARCHITECTURE

Lambda: separate batch and streaming pipelines, reconcile in serving layer. Complex, two codebases. **Kappa:** one streaming pipeline, replay from log for backfills. Modern. Mention you've seen both; Kappa is winning.

SCHEMA EVOLUTION & THE WIRE

Avro/Protobuf with a schema registry. Producers register schemas with compatibility modes (backward, forward, full). Without this, breaking

changes hit production. Confluent Schema Registry is the reference.

The "in flight to at rest" question

If they ask *"walk me through a play event from device to dashboard,"* here's the senior answer:

1. **Client SDK** emits event to a regional collector (Netflix uses something like Keystone — a Kafka cluster).
2. **Schema-validated** at the edge against a registry. Bad events go to a DLQ topic, not /dev/null.
3. **Multi-tier Kafka topics:** raw → validated → enriched. Each stage is a Flink job that adds metadata (geo lookup, profile join).
4. **Two consumers fork off:** a real-time consumer (recommendations, alerting — sub-second SLAs) and a batch consumer (S3/Iceberg sink for analytics).
5. **S3 lands raw events partitioned by event_date/hour.** Iceberg/Delta gives ACID and time-travel.
6. **Spark / dbt models** build the dimensional layer (the `fact_viewing_session` we modeled in §02) on a daily schedule.
7. **BI / Tableau / Superset** queries the modeled layer. Recommendation features re-publish to a feature store (Feast / EVCache).
8. **Quality gates** at every stage: row counts vs. expected, schema checks, freshness SLAs. Failures page on-call.

If you can narrate this fluidly with one or two trade-offs called out (e.g., "we used Iceberg over Delta because of multi-engine support"), you've passed the streaming bar.

§ 06 — DEEP-DIVE BANK

The technical deep-dive — questions to *rehearse* answers for.

After the coding section, expect 15–20 minutes of "tell me about a time you..." and "how would you design...". Have polished answers for these. Use the STAR format (Situation, Task, Action, Result) but compress to 90 seconds each.

From your resume — likely probes

- **"Walk me through a hard data-quality problem you solved."** Have one ready. Specify the symptom (e.g., row counts off by 0.3%), the diagnosis (late-arriving events from a misconfigured collector), the fix (watermark adjustment + backfill window), and the result (SLA met, alert added).
- **"Tell me about a time you owned a pipeline end-to-end."** Pipeline name, scale (rows/day, latency SLA), tech stack, what went wrong, how you fixed it.
- **"Describe a model you designed that you'd do differently today."** This is a self-awareness probe. Don't say "nothing." Pick one — wrong grain, over-normalized, missed an SCD requirement — and explain what you learned.
- **"How do you handle PII at scale?"** Tokenization, column-level encryption, separate access tiers, audit logging, GDPR delete cascades. You have direct experience here — lean in.

System-design lite — designs to rehearse

DESIGN A REAL-TIME RECOMMENDATION PIPELINE

Kafka events → Flink for feature aggregation → feature store (Feast/Redis) → model serving (Triton/TF-Serving). Discuss feature freshness SLA (sub-second) vs. batch features (daily). Cold-start handling.

DESIGN A DATA QUALITY FRAMEWORK

Layered checks: schema (registry), volume (row count vs. 7-day average ± stddev), distribution (KS test on key columns), referential integrity (FK orphans). Write to a monitoring topic; route critical failures to PagerDuty.

DESIGN GDPR DELETE-ON-REQUEST

Token mapping service: replace user_id with token at ingest. On delete, invalidate the token mapping — the events become un-rejoinable. Plus a propagation queue for downstream materialized views. SLA: 30 days.

DESIGN A/B EXPERIMENTATION INFRA

Assignment service (sticky bucketing on user_id hash), exposure logging (every served variant logged), metric pipeline (joins exposures to outcome facts), stats engine (CUPED + sequential testing). SRM monitor as a first-class citizen.

§ 07 — DAY-OF PLAYBOOK

The interview-day playbook.

The morning

- **Don't cram new material.** Re-read your own one-page cheat sheet (the seven SQL patterns, the modeling five-move sequence, the streaming vocabulary table).
- **Test your setup 30 min early.** Camera, mic, screen-share, CodeSignal login, scratch paper, water. Have a second laptop or phone ready in case of technical failure.
- **Have the recruiter's contact open in another tab.** If something breaks, you message them immediately.

In the round — micro-behaviors

- **First 30 seconds of every problem:** ask 1–2 clarifying questions. Even if the prompt seems complete. This is muscle memory you should drill.
- **Narrate everything.** "I'm going to start with a CTE that aggregates per user. Then I'll rank within user. Then I'll filter." If you go silent for 60+ seconds, you've lost signal.
- **If you're stuck:** say so. "I'm trying to remember the exact syntax for the date function in Snowflake — can I write it as `date_trunc` and we move on?" The interviewer will say yes 100% of the time.
- **If you finish early:** don't sit silently. Walk through edge cases out loud. "What if the input is empty? What if there are duplicate timestamps?" This is free senior signal.
- **Last 5 minutes:** always reserved for your questions. Have 3 ready (see next section).

If something goes wrong

- **You give a wrong answer and realize it 2 minutes later:** say so immediately. "Actually, I want to revise — the LEFT JOIN should be on the cohort, not the activity, otherwise we drop users with zero activity." This is a strong signal.
- **You don't know the answer at all:** don't fake. "I haven't worked with Iceberg specifically; I've used Delta. Can I tell you how I'd approach it from first principles, and you can correct me?"
- **You blank on syntax:** "In standard SQL, this is `X`. I'm forgetting the exact Snowflake function name — would you mind reminding me?" Asking is fine. Stalling is not.

---

§ 08 — YOUR QUESTIONS

Questions to ask *them*.

The questions you ask are scored. They reveal what you care about. Pick 2–3 from this list per round; tailor by the interviewer's role.

For an engineering interviewer

- "What does your team's batch-vs-stream split look like? Are most pipelines moving toward Kappa, or do you still run dual-mode?"
- "How does the team handle schema evolution across producers and consumers — registry-driven, or convention?"
- "What's the most painful operational issue your team has dealt with in the last quarter, and how did the team grow from it?"
- "How is data quality scored at Netflix? Is there a centralized framework, or does each team build its own?"

For a manager / hiring manager

- "What does success look like in the first 90 days for someone in this role?"
- "How does the team balance platform work versus product-aligned analytics?"
- "How autonomous are individual engineers in choosing tools and patterns? What's the line between team standards and personal preference?"
- "What's the team's current biggest unsolved problem, and what's blocking progress?"

About culture (Netflix specifically)

- "How does the 'context, not control' principle play out day-to-day on your team?"
- "How do you handle the tension between high-performing individuals and team cohesion in practice?"
- "What kinds of mistakes are tolerated, and which aren't?"

> ONE LAST THING
>
> The interview is also *your* evaluation of *them*. Listen for: how they answer hard questions (defensive vs. open), whether they admit gaps, whether the team sounds healthy. You're picking your next 2–4 years too.

5-DAY DRILL PACK · NETFLIX · STUDIO/CONTENT DE

The five days before the *conversation*.

Tuned for Sr/Staff Studio/Content. Five days, two-and-a-half hours per day, four weak areas. The point isn't volume — it's getting the same patterns through your fingers enough times that your hands type them while your mouth narrates trade-offs.

CONTENTS

§ 01 — THE DOMAIN

Studio/Content data is *not* Member/Growth data.

If you walk in expecting to talk about viewing events and DAU, you'll mismatch the interviewer's mental model. Studio/Content sits between the production process, the content catalog, the talent system, and finance. The data is SCD-heavy, multi-currency, hierarchical, and slow-moving in row count but high-stakes in correctness.

PRODUCTION LIFECYCLE

Greenlight → development → pre-production → production → post → QC → release → performance. Each stage has its own facts (deals signed, budget allocated, episodes shot, edit milestones, dub/sub completion). Time-to-stage and stage-cycle-time are the analytics.

TALENT & DEALS

Actors, directors, writers, crew. Hierarchical contracts (overall deal → series deal → episode fee). Royalty waterfalls. Residuals. Multi-party splits. SCD2 everywhere because deal terms are renegotiated.

CONTENT CATALOG

Title hierarchy (franchise → series → season → episode). Multi-region availability windows. Per-country ratings and certifications. Localization assets (audio, subtitle, key art) per language. Genre/mood/theme taxonomies that change.

PERFORMANCE ATTRIBUTION

Studio cares about: did our investment pay off? "Member value per dollar spent" is the holy-grail metric. This requires joining production cost data to viewing performance — and viewing has to be normalized for catalog effect (a title in 100M households has a head start over one in 10M).

FINANCE & ROYALTIES

Royalty calc by territory by license window. Currency conversion (multi-currency facts, FX rates as a dim). Amortization schedules (a show's cost is amortized over its expected lifetime — viewing pattern affects the amortization curve). This will come up.

THE UNIQUE VOCABULARY

"Slate," "greenlight," "pickup," "license window," "first-run," "library title," "tentpole," "originals vs licensed," "P&A" (prints & advertising), "residuals," "MFN" (most-favored-nation clause). Use these terms when modeling. Demonstrates context.

THE SR/STAFF DIFFERENTIATOR

At Sr/Staff level for Studio/Content, you're expected to think about **data products that influence greenlight decisions and renewal decisions** — not just dashboards. When asked "design a model for X," extend the answer to "...and here's what insight this unlocks for the studio leadership team." That framing alone lifts you a level.

§ 02 — 5-DAY PLAN

The plan, by the day.

~2.5 hours per day. Not negotiable: do the timed drills. The point isn't to find the answer — it's to get used to the clock running.

1 FOUNDATION · MODELING ~2.5 hrs

Modeling muscle memory.

30 MIN · READ

The five-move sequence

Re-read §02 of the main notebook. Practice saying the five moves out loud. Write each one on a sticky note next to your monitor. **Goal:** by end of week, you can recite them in 15 seconds.

60 MIN · DRILL

Modeling drills #1 and #2 (timed, 25 min each)

From §03 below: *Production lifecycle* and *Talent & royalties*. Use a real timer. Whiteboard, paper, or Excalidraw. Don't peek at the solution until the timer rings. After: compare yours to the worked answer, note what you missed.

30 MIN · VOCABULARY

Review the Studio/Content vocabulary above. Look up any terms you can't define crisply (P&A, MFN, residuals, slate, tentpole). Write 1-line definitions in your own words.

30 MIN · REFLECT

Did you ask clarifying questions before drawing tables? Did you state grain explicitly? Did you volunteer two failure modes? If not — these are your reps for tomorrow.

2 SQL · WINDOW FUNCTIONS & GAPS/ISLANDS ~2.5 hrs

SQL fluency through repetition.

45 MIN · DRILL

SQL drills #1, #2, #3 (timed, 15 min each)

Solve at your own SQL playground or sqlfiddle. Time yourself. Don't just read the solutions — type them.

30 MIN · PATTERN REPS

Type out the gaps-and-islands template (LAG → flag → cumulative SUM) **three times from memory**. By rep three, your fingers should know it.

45 MIN · DRILL

SQL drills #4 and #5 (timed, 20 min each)

These are the harder ones — multi-CTE, finance-flavored. Talk out loud while solving (practice for the live round).

30 MIN · READ & REVIEW

Read §03 patterns 1, 2, 6 of the main notebook. Note the `QUALIFY` shortcut (Snowflake) — Netflix uses Snowflake heavily. Mention it in the round.

3 PYTHON · CODESIGNAL PACE ~2.5 hrs

Coding under a clock.

30 MIN · WARM UP

Type-from-memory templates

Open a blank file. Type out, from memory: (a) sessionization in Python, (b) top-K with heap, (c) sliding-window aggregator. These are the three patterns that cover most CodeSignal problems.

75 MIN · DRILL

Python drills #1, #2, #3 (timed, 25 min each)

Use a fresh editor (not your IDE — closer to CodeSignal feel). 25 min hard cap each. **Narrate while coding.** Record yourself if helpful — listen back to spot mumbling/long silences.

30 MIN · SOLVE AGAIN, OPTIMIZE

Take the slowest of your three solutions and rewrite it for better complexity. Practice the verbal arc: "My first pass was O(n²); I see I can use a hash map to get O(n)..."

15 MIN · REFLECT

Where did the clock catch you? Off-by-ones? Edge cases? Forgetting what `defaultdict(int)` does? Note one thing to drill tomorrow morning.

4 STREAMING · DEEP-DIVE PREP ~2.5 hrs

Vocabulary becomes fluency.

45 MIN · DRILL

Streaming concept Q&A (§06 below)

Read each prompt. **Answer out loud, on a timer (90 seconds each).** Then read the model answer. Goal: by end of week, the words "watermark," "exactly-once," "backpressure," "Kappa" come out without effort.

45 MIN · MOCK

Trace a play event from device to dashboard

Whiteboard or doc. 8 stages, 30 seconds each. Out loud. Repeat 3 times until it flows. This is the "narrate the architecture" question that almost always comes up in deep-dives.

30 MIN · RESUME REHEARSAL

Pick three projects from your resume. Write a 90-second STAR answer for each. Rehearse out loud. **Reminder per memory edits:** don't volunteer your specific employer, title, or founder role unless asked directly — describe the work.

30 MIN · CATCH-UP

Re-do any drill from days 1-3 you didn't finish. Or re-do the one you got wrong.

5 INTEGRATION · CALM ~2 hrs

Light reps, mental rehearsal, sleep.

30 MIN · ONE-PAGE REVIEW

Read §07 (cheat sheet) twice. Don't drill new material. Trust the reps you've put in.

30 MIN · ONE MOCK END-TO-END

Pick one modeling prompt + one SQL + one Python. 15 min each. Light pace, not a stress test. Confirm the muscle memory is there.

30 MIN · LOGISTICS

Test camera, mic, screen-share. Clear desktop. Have water ready. Have the recruiter's number open. Confirm timezone of the interview.

30 MIN · THE 3 QUESTIONS

Pick your three questions to ask each interviewer (from §08 of the main notebook). Write them on paper next to your monitor. Different questions for engineer vs manager.

THEN · STOP

No drills the night before. Walk, eat, sleep. The reps are already in. The interview is a conversation.

Five modeling prompts. *Whiteboard. Timer.*

Each prompt is sized for 25 minutes. Don't peek. Use the five-move sequence: clarify → grain → conceptual → physical → stress test. Whiteboard or paper is fine; type only if you're going to type during the real round.

MODEL · 01 PRODUCTION LIFECYCLE · 25 MIN

"Design a data model that lets the studio leadership team see, for any title in production, where it is in the pipeline (greenlight → development → pre-prod → production → post → QC → release) and how long it's been at each stage. They want to spot stuck projects."

▸ WALK-THROUGH

MODEL · 02 TALENT & ROYALTIES · 25 MIN

"Design a model for talent contracts and royalty payouts. An actor can have an overall deal with Netflix that covers multiple shows; within each show they have per-episode fees plus a backend (residuals, success bonuses). The legal team wants to be able to compute total liability per actor at any point in time, and finance wants to forecast next-quarter payouts."

▸ WALK-THROUGH

MODEL · 03 RENEWAL DECISIONS · 25 MIN

"Design a model that supports the renew/cancel decision for a series after season N has been released. The studio team wants a single 'renewal scorecard' per series with: total viewing, viewing per dollar spent, cost amortization status, talent renewal cost, and comparable shows."

▸ WALK-THROUGH

MODEL · 04 LOCALIZATION · 20 MIN

"Design a model for localization assets — every title needs dub and subtitle assets per language, each with its own QC state (in progress, in review, approved, rejected). The localization team needs to see which titles are blocking which territory launches."

▸ WALK-THROUGH

MODEL · 05 SLATE PLANNING · 25 MIN

"Design a model that supports the studio's annual slate planning — they need to see, for any future quarter, the expected mix of releases (genre, budget tier, target territory, talent tier) and how that compares to performance benchmarks of past slates. Help them spot gaps (e.g., 'we're under-indexed on family content in Q3')."

▸ WALK-THROUGH

§ 04 — SQL DRILLS

Five SQL drills. Type. Don't *read*.

These problems target your weak areas: window functions, gaps/islands, sessionization. All flavored for Studio/Content. Type each solution. Time-boxed. Compare to the model answer only after.

THE SCHEMA YOU'RE WORKING WITH

Assume tables: `dim_title(title_id, series_id, season_id, ep_num, runtime_sec, genre, release_date)`, `fact_viewing_session(session_id, profile_id, title_id, session_start_ts, watch_seconds, qualified_view_flag, country_code)`, `fact_production_spend(title_id, spend_date, amount_usd, category)`, `fact_payout(payout_id, talent_id, title_id, deal_id, payout_date, amount_usd)`.

SQL · 01 WINDOW · 15 MIN

For each series, find the episode in each season that had the highest 28-day completion rate. Return series_id, season_id, ep_num, completion_rate.

▸ SOLUTION

SQL · 02 GAPS & ISLANDS · 20 MIN

A "binge sequence" for a profile is a run of consecutive episodes within the same series watched within 24 hours of

each other (in release order). Given `fact_viewing_session`, identify each binge sequence and return profile_id, series_id, sequence_start_ts, sequence_end_ts, episode_count.

▸ SOLUTION

SQL · 03 WINDOW · LAG · 15 MIN

For each title, calculate the "second-week drop-off" — the percentage decrease in qualified views from the first 7 days post-release to days 8-14. Return title_id, week1_views, week2_views, drop_pct.

▸ SOLUTION

SQL · 04 MULTI-CTE · FINANCE · 25 MIN

Calculate "viewing efficiency" per title in its first 90 days: total qualified views divided by production cost (USD). Return the top 10, but exclude titles with less than $1M production cost (too noisy) and less than 90 days since release.

▸ SOLUTION

SQL · 05 SCD2 QUERY · 20 MIN

Given `dim_deal(deal_id, talent_id, title_id, base_fee_usd, effective_from, effective_to)`

with SCD2 (effective_to NULL = current), compute total contracted fee per talent as of any given date `D`. Write a query parameterized by `:as_of_date`.

▸ SOLUTION

§ 05 — PYTHON DRILLS

Three problems. *25 minutes* each. Type out loud.

CodeSignal-style. The trick isn't the algorithm — it's getting through the boilerplate (parsing input, defaultdicts, edge cases) without hesitation. Set a timer. Talk while you type, like you would in the live round.

CODE · 01 SESSIONIZATION · 25 MIN

Given a list of viewing events `[(profile_id, title_id, ts, watch_seconds), ...]` (any order), group consecutive events for the same profile into "viewing sessions" where the gap between events is ≤ 30 minutes. Return a list of sessions: `[(profile_id, session_start, session_end, total_watch_seconds, distinct_titles)]`.

▸ SOLUTION

CODE · 02 SLIDING WINDOW · HEAP · 25 MIN

Implement a real-time "trending titles" tracker. Events arrive in chronological order: `(ts, title_id, watch_seconds)`. At any point you must be able to return the top K titles by watch_seconds *in the last hour*. Implement `add_event` and `top_k` with reasonable performance.

▸ SOLUTION

CODE · 03 HASH · AGGREGATION · 25 MIN

You have `views: List[(profile_id, title_id, watch_seconds)]` and `cohorts: Dict[profile_id, str]` mapping profiles to a cohort label (e.g., "new", "returning", "churned"). Compute, for each (cohort, title_id), total watch_seconds and unique viewer count. Return as a sorted list of `(cohort, title_id, watch, viewers)` sorted by watch descending.

▸ SOLUTION

The CodeSignal warm-up routine

The 5 minutes before the timer starts in CodeSignal: read the problem twice. Identify input/output types. Mentally run the example by hand. **Decide on data structures before typing.** "I need fast lookup — dict. I need ordered iteration — sort first. I need top-K — heap." 30 seconds of structure-picking saves 5 minutes of debugging.

§ 06 — STREAMING Q&A

Streaming concepts — answer each *out loud*.

90 seconds per answer. Talk to the wall. Then read the model answer. Repeat any you fumbled.

STREAM · 01

"What's the difference between event-time and processing-time? Why does it matter?"

▸ MODEL ANSWER (≈ 75 SEC)

STREAM · 02

"What is a watermark in Flink? How do you choose its value?"

▸ MODEL ANSWER

STREAM · 03

"Explain exactly-once semantics. How does Flink achieve it?"

▸ MODEL ANSWER

STREAM · 04

"What's the difference between Lambda and Kappa architectures? Which would you pick today?"

▸ MODEL ANSWER

STREAM · 05

"How do you handle schema evolution in a streaming system?"

▸ MODEL ANSWER

STREAM · 06

"Walk me through how a play event travels from a user's TV to the executive dashboard."

▸ MODEL ANSWER (THE 8-STEP NARRATIVE)

STREAM · 07

"What's backpressure? How do streaming systems handle it?"

▸ MODEL ANSWER

STREAM · 08

"Tumbling, hopping, and session windows — when do you use each?"

▸ MODEL ANSWER

§ 07 — ONE-PAGE CHEATSHEET

The page you read on day 5.

Print this. One side. Read it twice the morning of the interview, then put it away.

Modeling — the five moves

MOVE	WHAT YOU SAY / DO
1. Clarify	3-5 questions before drawing. Use cases, SLA, retention, key stability, time zone.
2. Grain	One sentence per fact: "One row per X per Y per Z." Out loud.
3. Conceptual → physical	Entities first. Then partitioning, clustering, SCD strategy.
4. SCDs & time	Type 1 (overwrite), Type 2 (versioned rows), Type 6 (hybrid). Justify.
5. Stress test	Volunteer 2 failure modes. Late data, GDPR, schema evolution, hot partitions.

SQL — the seven patterns

PATTERN	SHAPE

`Top-N per group`	ROW_NUMBER() OVER (PARTITION BY g ORDER BY metric DESC) → WHERE rn ≤ N
`Gaps & islands`	LAG → flag boundary → cumulative SUM = session_id → GROUP BY
`Funnel`	MAX(CASE WHEN step='X' THEN 1 END) per user → ratios with NULLIF
`Dedup`	ROW_NUMBER PARTITION BY natural_key ORDER BY ingest_ts DESC = 1
`Cohort retention`	cohort_week × activity_week LEFT JOIN, weeks_since = DATEDIFF
`Running total`	SUM() OVER (ORDER BY … ROWS UNBOUNDED PRECEDING)
`HLL / approx`	APPROX_COUNT_DISTINCT — mention it for scale

Python — the five reflexes

NEED	REACH FOR
`Group + aggregate`	collections.defaultdict(int / list / set)
`Top-K`	heapq.nlargest, or manual min-heap of size K
`Sliding window`	collections.deque + running totals dict
`Sessionization`	sort within group, walk pairs, track open session, append on close
`O(1) cache`	collections.OrderedDict, move_to_end / popitem(last=False)

Streaming — vocabulary to drop

TERM	ONE-LINER
`Watermark`	"Promise: all events with ts ≤ W have been seen."
`Event-time`	"When it happened (client clock); not when we processed it."
`Exactly-once`	"Checkpoint + transactional sinks. Expensive. Most pipelines use at-least-once + idempotent writes."
`Backpressure`	"Downstream slower than upstream — system pauses upstream automatically."
`Kappa`	"One streaming pipeline. Backfills via log replay. Default today."
`Schema registry`	"Centralized schema validation; backward/forward compatibility modes."
`Tumbling vs. hopping vs. session`	"Non-overlapping vs. overlapping fixed-size vs. gap-defined dynamic."

Studio/Content vocabulary

TERM	MEANING
`Slate`	Planned set of releases for a period
`Greenlight`	Formal approval to begin production
`Tentpole`	High-budget flagship release
`P&A`	Prints & Advertising — marketing spend

`Residuals`	Ongoing payments to talent post-release
`MFN clause`	Most-favored-nation: my terms match the best terms granted to anyone else
`Amortization`	Spreading content cost over expected useful life
`License window`	Period during which a title is available in a territory

The 90-second answer template (for deep-dive STAR)

1. **Situation** (15s): "We had a [type of] pipeline running [scale]; the issue was [observable symptom]."
2. **Task** (15s): "I needed to [decision / outcome you owned]."
3. **Action** (45s): "I [diagnosed by X], decided to [Y because Z trade-off], implemented by [details]."
4. **Result** (15s): "[measurable outcome] within [timeframe]; [what changed structurally]."

What to do in the first 30 seconds of every problem

1. Restate the problem in your own words.
2. Ask 1-2 clarifying questions, even if obvious.
3. State assumptions out loud.
4. Walk through one example by hand.
5. Then start writing.

> THE SINGLE BIGGEST SIGNAL
>
> **Narrate. Constantly.** "I'm using a CTE here because the alternative is repeating this aggregation." "I'm picking ROW_NUMBER over RANK

> because ties shouldn't both qualify." Your interviewer cannot give you a senior score for code they only saw appear in silence.
